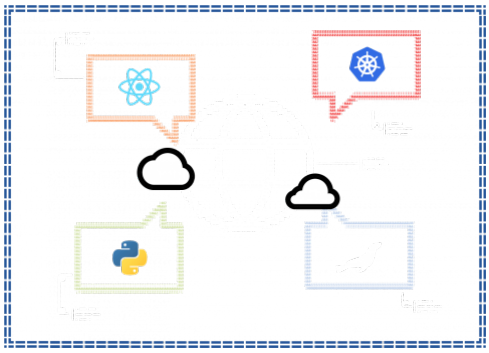
Learn Kubernetes in context

Learn Kubernetes using Red Hat Developer Sandbox for OpenShift

Red Hat Developer Sandbox for OpenShift ("Sandbox") is a great platform for learning and experimenting with Red Hat OpenShift. Because OpenShift is built on Kubernetes, the Sandbox is also a great platform for learning and experimenting with Kubernetes.

This activity takes you through the creation of an application using plain Kubernetes instead of OpenShift.

Get access to the sandbox in minutes



Red Hat OpenShift Sandbox ("Sandbox") is a great platform for learning and experimenting with Red Hat OpenShift. Because OpenShift is built on Kubernetes, the Sandbox is also a great platform for learning and experimenting with Kubernetes. This article takes you through the creation of an application using plain Kubernetes instead of OpenShift.

Prerequisites:

1. An OpenShift Sandbox account
2. The Kubernetes command line interface (CLI), `kubectl`, installed on your local PC
3. git installed on your local PC
4. An image registry that you can use – quay.io is a good option.
5. The ability to build an image on your local PC using either podman or docker

Here's what we'll be doing:

- Logging into the sandbox
- Creating a backend program called "quotes", including setting environment variables to be used by the code.
- Creating a React.js frontend program called "quotesweb"
- Viewing quotesweb in our browswer
- Scaling backend to two pods and observing the result in quotesweb

- Creating a Persistent Volume Claim to support MariaDB running in Kubernetes
- Creating a Secret to be use with the database
- Creating a MariaDB database, "quotesdb", running in Kubernetes
- Creating and populating the table "quotes" in the "quotesdb" database.
- Destroying the MariaDB pod to observe Kubernetes' "Self Healing"
- Updating the backend program "quotes" to Version 2 and observing the results in "quotesweb".

The Kubernetes features used, as described at the Kubernetes by Example web site, will be:

- pods
- labels
- deployments
- services
- service discovery
- environment variables
- namespaces
- volumes
- persistent volumes
- secrets
- logging

Personal note from the author

Sit down, relax, and be prepared to spend some quality time with this tutorial. We cover a lot of ground, and I've attempted to mimic a real-life situation in order to bring the most value to your time spent. If you're new to Kubernetes, you'll go from zero to deploying applications in this guide. You'll be rolling out a backend application, a database, and a frontend application. You'll also be scaling an application and updating another application. This is hands-on with skills that are 100 percent applicable to a production environment. Thanks for taking to time to trust me with your learning. Enjoy.

Also, when commands are referenced inline, they are shown in a different typeface, e.g. `kubectl config view`.

Preparation

You will need to download or clone the following three repositories (repos) from Github.com using the following commands:

```
git clone https://github.com/DonSchenck/quotesweb.git
```

```
git clone https://github.com/DonSchenck/quotemysql.git
```

```
git clone https://github.com/DonSchenck/qotd-python.git
```

For the purposes of this tutorial, the three directories you created will be referenced by their repo name. Where you put them on your local machine is up to you.

Logging into the Sandbox

You don't actually "log into" a Kubernetes cluster. Instead, you set your local environment to connect to your cluster when issuing kubectl commands. This part is a bit cumbersome, but it's necessary. You can, of course, automate it. You can also use tools to help you moving forward. Finally, if you have the OpenShift CLI (oc) installed (not necessary for this tutorial), you can cheat and use the `oc login` command. If you can install the oc CLI, it makes life a lot easier.

There are three parts in play here:

1. Your credentials
2. A kubernetes (or OpenShift) cluster
3. A context, i.e. namespace within the cluster

After establishing those three parts, you use the context you desire.

The following commands will take care of this, but first we need to extract some pieces of information from our Sandbox. We'll need to get the following items:

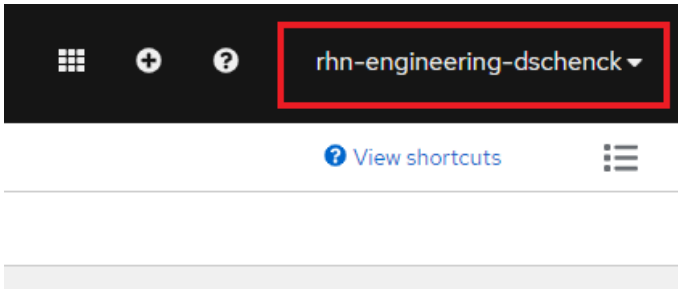
1. Username, which is represented by {username} in the following commands
2. Authorization token {token}
3. Name of the cluster {cluster_name}
4. Context assigned to us {context}
5. URL of the cluster API server {api_server_url}

You'll need to log into your Sandbox dashboard to get this information.

Username

Our value for {username}.

This is displayed in the upper right corner of the dashboard. For example:



Given this example, the username would be **rhn-engineering-dschenck**.

Of note: The namespace we'll be using is simply your username with "-dev" appended to it, e.g. **rhn-engineering-dschenck-dev**.

Authorization token

Our value for {token}.

If you click on the username and select "Copy login command" and log in as DevSandbox, you can see your token. If this is confusing, there is an article that will help.

Name of the cluster

Our value for {cluster_name}.

The cluster name is a modification of the host URL with all periods converted to dashes. Also, the "console-openshift-console-apps" portion of the host URL is swapped out for the API server. For example, if you navigate to the Topology page of your dashboard, your URL looks something such as this:

```
https://console-openshift-console.apps.sandbox.x8i5.p1.openshiftapps.com/topology/ns/rhn-engineering-dschenck-dev?view=graph
```

Given this, the cluster name will be **api-sandbox-x8i5-p1-openshiftapps-com:6443**.

Context assigned to us

Our value for {context}.

The context is constructed by combining your username with the name of the cluster in the following format: {username}-dev/{cluster_name}/{username}.

For example, using what we have up to this point, the value for {context} would be:

rhn-engineering-dschenck-dev/api-sandbox-x8i5-p1-openshiftapps-com:6443/rhn-engineering-dschenck

URL of the cluster API server

Our value for {api_server_url}.

This is almost the same as the cluster name, but it keeps the periods. For example, given what we have above, the URL would be :

https://api.sandbox.x8i5.p1.openshiftapps.com:6443

Viewing and deleting your local PC Kubernetes configuration

The command `kubectl config view` will show you your configuration. If you wish, you can remove all of your Kubernetes local configuration information by deleting the file "~/.kube/config", e.g. `rm ~/.kube/config`.

Connecting to our Kubernetes cluster

Using the information we have, use the following commands — of course, substituting your own values where noted.

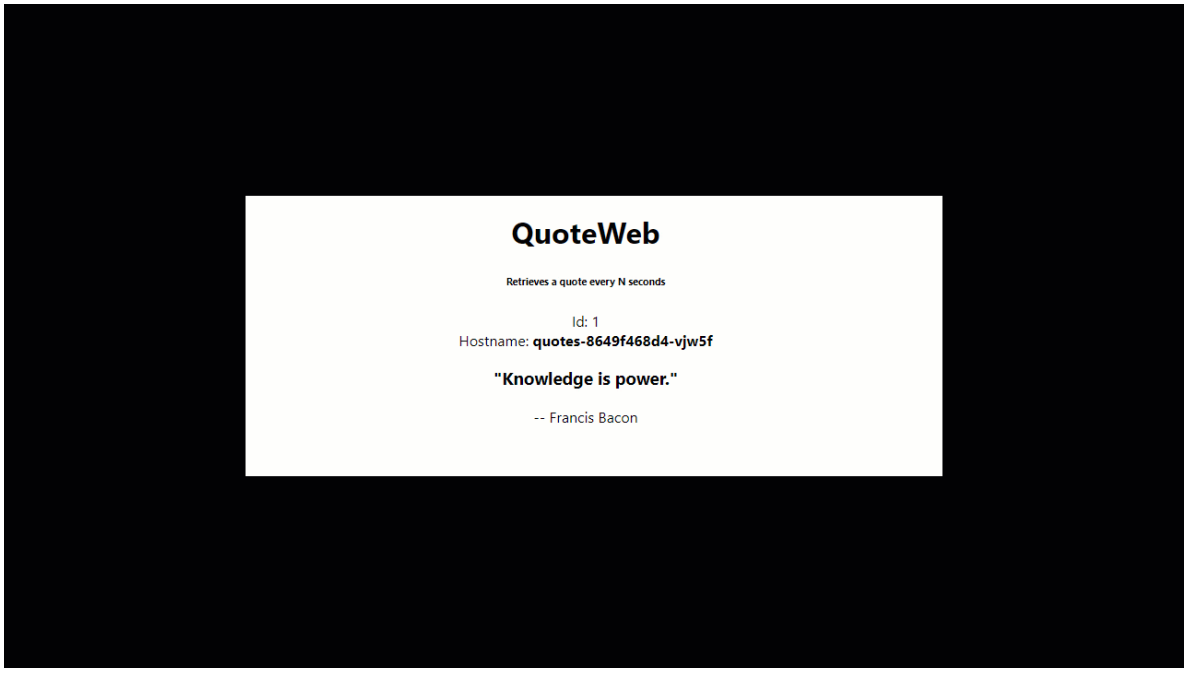
```
kubectl config set-credentials {username}/{cluster_name} --token={token}
kubectl config set-cluster {cluster_name} --server={api_server_url}
kubectl config set-context {context} --user={username}/{cluster_name} /
  --namespace={username}-dev --cluster={cluster_name}
kubectl config use-context {context}
```

Copy snippet

What we're creating in this tutorial

This tutorial will guide you through using Kubernetes to create three components:

- Backend RESTful service
- Frontend ReactJS web page
- MariaDB database



About the backend RESTful service

A backend application, "quotes", written in Python 3.8, supplies "Quote Of The Day"-type data via a RESTful API. The endpoints are as follows:

URL: **/**

METHOD: **GET**

Returns the string "qotd" to simply identify the service.

URL: **/version**

METHOD: **GET**

Returns a string denoting the version id of the service, e.g. "2.0.0".

URL: **/writtenin**

METHOD: **GET**

Returns the programming language in which the service is written. In this case, it is "Python", but this same service is available in several different programming languages.

URL: **/quotes**

METHOD: **GET**

Returns a JSON array of all of the quotes.

URL: **/quotes/random**

METHOD: **GET**

Returns a JSON object of one random quote from among the set of available quotes.

URL: **/quotes/{id}**

METHOD: **GET**

Returns a JSON object of one specific quote within the set of available quotes.

Creating the backend app (quotes)

In this step, we will create Kubernetes objects associated with the "quotes" application: a Deployment, a Service, and a Route (which is similar to the Ingress and Ingress Controller objects). We will also set an Environment Variable that will allow us to change the name of the database service if we want to.

About Route, Ingress, and Ingress Controller

Because Red Hat Developer Sandbox for OpenShift is administered by Red Hat, you do not have administrator access to the Kubernetes cluster. One of the limitations of this is that you are not granted rights to create Ingress and Ingress Controller objects.

OpenShift has its own built-in Ingress-like object, the Route. For this tutorial, we're going to "cheat" and use the Route object. Be aware that we are using this workaround; in your own Kubernetes cluster you'll want to create the Ingress and Ingress Controller objects.

In the directory where you cloned the qotd-python repo, move into the "k8s" sub-directory and run the following three commands:

```
kubectl create -f quotes-deployment.yaml
```

Copy snippet

```
kubectl create -f service.yaml
```

Copy snippet

```
kubectl create -f route.yaml
```

Copy snippet

For example:

```
dschenck@DSCHENCK-W10E ~\src\github\donschenck\qotd-python\k8s v2 +0 ~2 -0 !
kubectl apply -f .\quotes-deployment.yaml
deployment.apps/quotes created
```

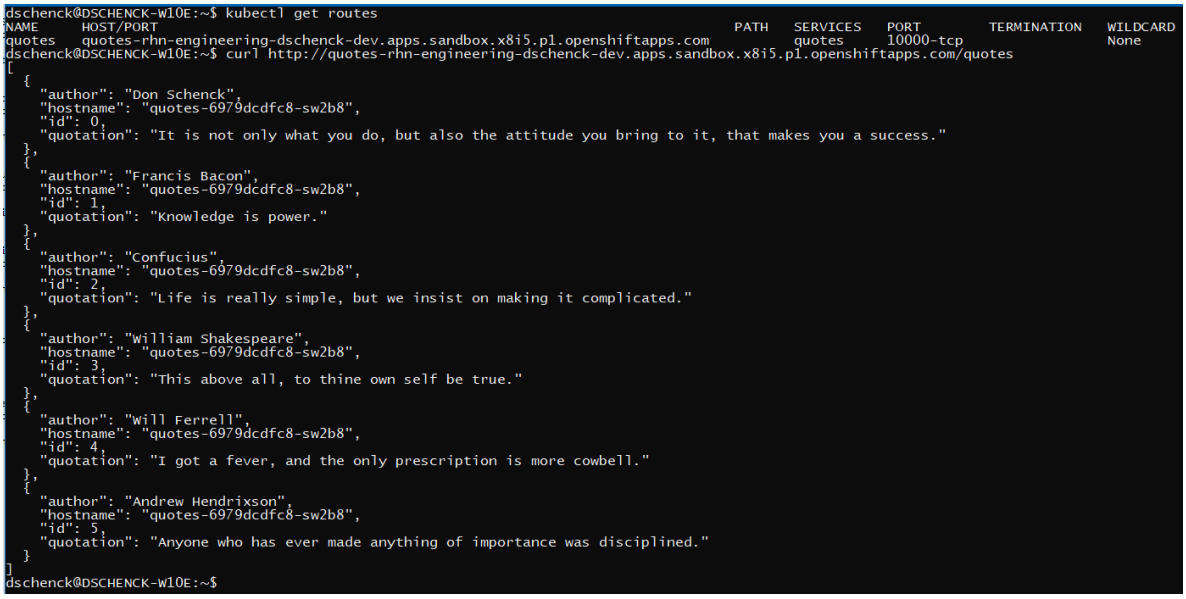
```
dschenck@DSCHENCK-W10E ~\src\github\donschenck\qotd-python\k8s v2 +0 ~2 -0 !
kubectl apply -f .\service.yaml
service/quotes created
```

```
dschenck@DSCHENCK-W10E ~\src\github\donschenck\qotd-python\k8s v2 +0 ~2 -0 !
kubectl apply -f .\route.yaml
route.route.openshift.io/quotes created
```

At this point we have the backend "quotes" application running in a pod. It's exposed within Kubernetes as a Service, and the Route allows anyone to access it over the internet.

If you want, you can run the command `kubectl get routes` and then use the `curl` command with the "/quotes" route URL to see the service serving up data. Here's a Linux example:

(The PowerShell equivalent is `(curl http://quotes-rhn-engineering-dschenck-dev.apps.sandbox.x8i5.p1.openshiftapps.com/quotes).content.`):



pods and labels

When you create the deployment, Kubernetes will pull the image from the image registry named in the YAML file and will create a pod. It will also assign the labels which you specified in the deployment.

The pod name is automatically generated from the deployment name, with random characters appended to it.

Viewing the contents of the file quotes-deployment.yaml, we can see that the pods will be named "quotesweb" (plus the random characters, e.g. "quotesweb-5468c95fc6-5sp9j"), while the label will be "app=quotesweb".

```
kind: Deployment
apiVersion: apps/v1
metadata:
  name: quotesweb
  labels:
    app: quotesweb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: quotesweb
  template:
    metadata:
      labels:
        app: quotesweb
    spec:
      containers:
        - name: quotes
          image: quay.io/donschenck/quotesweb:v1
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
              protocol: TCP
```

Copy snippet

Note that the pod name and app name *do not* need to be the same. Be careful here; this is where good management or poor management can a big difference.

Looking at the YAML file, we can see that the deployment (quotes-deployment.yaml) will use the following image:

quay.io/donschenck/quotes:v1

This is a Linux image that has data (six "Quote Of The Day"-type entries) hard-coded into the source code. Later, we'll upgrade to version 2, which reads quotes from a MariaDB database (which will also be running in our Kubernetes cluster).

Creating the frontend app (quotesweb)

Before we create the React.js frontend program, we need to change some code, build an image, and push the image to a publicly available registry from which we can pull it into our Kubernetes cluster.

The first thing we need to do is change the source code for the quotesweb application to point to the Route we created to the "quotes" service running in the Kubernetes cluster. We can find the URL of this route by running the following command:

```
kubectl get routes
```

Copy snippet

For example:

```
→  ~ kubectl get routes
NAME      HOST/PORT                                                PATH   SERVICES   PORT       TERMINATION   WILDCARD
quotes    quotes-rhn-engineering-dschenck-dev.apps.sandbox.x8i5.p1.openshiftapps.com          quotes     10000-tcp                None
```

The URL, plus the endpoint we need ("/quotes/random"), needs to be used in the "quotesweb" application. For example, a URL like the following:

```
http//quotes-rhn-engineering-dschenck-
dev.apps.sandbox.x8i5.p1.openshiftapps.com/quotes/random
```

To do this, move into your "quotesweb/src/components" directory and change the file "quotes.js". Substitute your URL in the following line of code (it's line 26):

```
fetch('your-url-goes-here')
```

```
getData = () => {
    fetch('http://quotes-rhn-engineering-dschenck-dev.apps.sandbox.x8i5.p1.openshiftapps.com/quotes/random')
        .then(data => {
            this.setState({quote:data});
            // call getData() again in N seconds
            this.intervalID = setTimeout(this.getData.bind(this), 2000);
        });
}
```

This is the line you need to change

Save this change.

Move back into your "quotesweb" directory where the file "Dockerfile" is located, and build your image. You will need to use your own naming pattern based on your own image registry. For example, if you are using docker.io as your image registry, and your username there is "janedoe", you'd use the following docker or podman command:

`docker build -t docker.io/janedoe/quotesweb:v1 .`

`podman build -t docker.io/janedoe/quotesweb:v1 .`

Here's the exact command I used (I used podman and Podman Desktop for my development). I'm using Red Hat's quay.io image registry:

```
podman build -t quay.io/donschenck/quotesweb:v1
```

Copy snippet

No matter what image registry you use, you'll need to log in to it, e.g. `docker login`.

With the image built, push it to your image registry. For example:

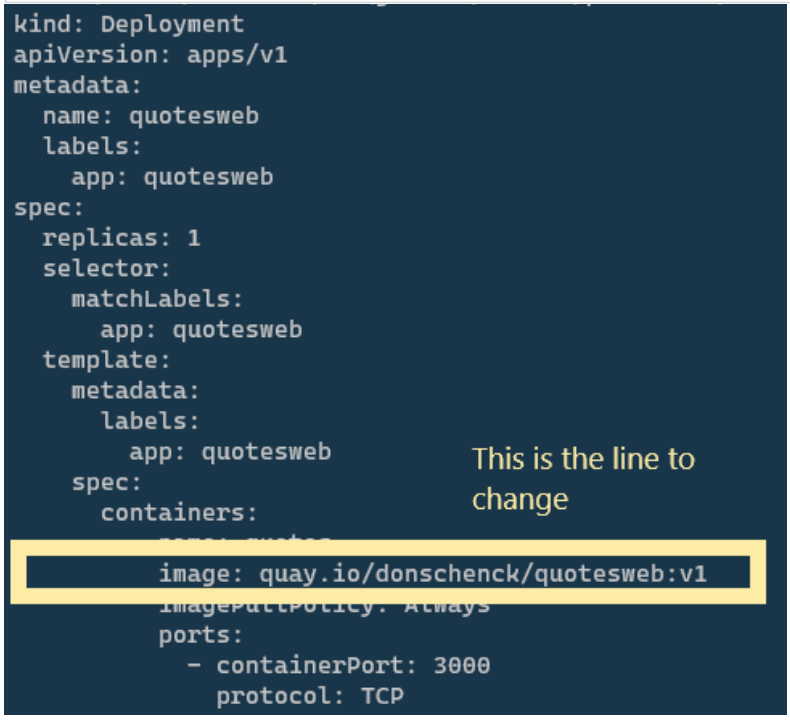
```
podman push quay.io/donschenck/quotesweb:v1
```

Copy snippet

The name of the image we create (e.g. "quay.io/donschenck/quotesweb:v1") will be used when we alter the deployment file, "quote-deployment.yaml". This deployment file is in the "k8s" subdirectory of "quotesweb". Find and change the following line, replacing the image name with your own image.

Optionally, you can leave it as-is and use the image that I've built.

```
image: quay.io/donschenck/quotesweb:v1
```



This change will direct Kubernetes to pull your custom-built image in order to create the "quotesweb" frontend application.

Why use the external, publicly available route?

When you want one service in Kubernetes to communicate with another Kubernetes service, you use the internal service name. For example, the URL to communicate with the "quotes" service might be as follows: "http://quotes/quotes/random". But, because we are using a React.js application, this won't work. Remember: React works by sending a payload of JavaScript to the browser, where it executes. Because this JavaScript code, which is communicating with the RESTful API "quotes", is running *outside* of Kubernetes (i.e. in the browser), it must use a public URL to reach our backend ("quotes") service.

"But can't the whole world access this?"

Yes. If this were your actual production architecture, this is where you would implement a form of Authorization and/or use an API service (such as 3scale).

Time to get our front end "quotesweb" application up and running in our Kubernetes cluster.

In your quotesweb/k8s directory on your local machine, run the following three commands to create the Deployment, the Service, and the Route:

```
kubectl create -f quotesweb-deployment.yaml
```

Copy snippet

```
kubectl create -f quotesweb-service.yaml
```

Copy snippet

```
kubectl create -f quotesweb-route.yaml
```

Copy snippet

```
[→  k8s git:(main) ✗ kubectl apply -f quotesweb-deployment.yaml
deployment.apps/quotesweb created
[→  k8s git:(main) ✗ kubectl apply -f quotesweb-service.yaml
service/quotesweb created
[→  k8s git:(main) ✗ kubectl apply -f quotesweb-route.yaml
route.route.openshift.io/quotesweb created
```

Viewing quotesweb in our browser

```
kubectl get routes
```

Copy snippet

```
→  k8s git:(main) ✗ kubectl get routes
NAME        HOST/PORT
CARD
quotes      quotes-rhn-engineering-dschenck-dev.apps.sandbox.x8i5.p1.openshiftapps.com
quotesweb   quotesweb-rhn-engineering-dschenck-dev.apps.sandbox.x8i5.p1.openshiftapps.com
```

Use the route for quotesweb; paste that into your browser.



Scale to meet demand

At this point, we have two apps (or Kubernetes services) running in our cluster. As you watch the quotesweb application in your browser, you will notice that the hostname is always the same. That's because we have one pod running our "quotes" service. We can prove this with the following command:

`kubectl get pods`

```
[→  k8s git:(main) ✗ kubectl get pods
NAME                         READY   STATUS    RESTARTS   AGE
quotes-8649f468d4-289h6      1/1     Running   0          17m
quotesweb-5468c95fc6-2xngb   1/1     Running   0          7m44s
```

While Kubernetes can be configured to auto-scale (by spinning up additional pods), we can mimic this behavior from the command line and observe the results in our browser. As we increase the number of pods, you'll notice that there are multiple hosts serving quotes. Use the following command to increase the number of pods to three:

```
kubectl scale deployments/quotesweb --replicas=3
```

Copy snippet

The next section of this tutorial will switch out the hard-coded quotes for quotes stored in a MariaDB database.

Creating a Persistent Volume Claim

While many Kubernetes database solutions offer an ephemeral option, that won't suffice for us. We need to make sure the database files remain intact even when the pods running MariaDB are deleted. This requires a Persistent Volume Claim, or PVC.

In your quotesmysql directory you'll find the file "mysqlvolume.yaml". We're creating a PVC with the name "mysqlvolume", and it's 5 GB in size, using the host file system. This is where we'll direct the MariaDB app to place the data files it needs.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysqlvolume
spec:
  resources:
    requests:
      storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
```

Copy snippet

Create the PVC by running the following command:

```
kubectl create -f mysqlvolume.yaml
```

Copy snippet

```
➜  quotemysql git:(main) kubectl apply -f mysqlvolume.yaml
persistentvolumeclaim/mysqlvolume created
➜  quotemysql git:(main) kubectl get pvc
NAME          STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
mysqlvolume   Pending                                      gp2            2m3s
```

Creating a Secret to be used to create the database.

Navigate to the "quotemysql" on your local PC. You'll find the file "mysql-secret.yaml".

```
apiVersion: v1
kind: Secret
metadata:
  name: mysqlpassword
type: Opaque
```

```
data:
  password: YWRtaW4=
```

Copy snippet

A note about the password: This is a Base64-encoded ASCII string from the value "admin".

In Linux, you get this value by running:

```
echo -n 'my-string' | base64
```

Copy snippet

For PowerShell, see the contents of the file "create_encoded_password.ps1".

```
$p = 'admin'
$b = [System.Text.Encoding]::ASCII.GetBytes($p)
$e =[Convert]::ToBase64String($b)
"Encoded password ($p) is: $e"
```

Copy snippet

Create the secret by running the following command:

```
kubectl create -f mysql-secret.yaml
```

Copy snippet

Creating a MariaDB database app

In your quotemysql directory, you'll find the file "mysql-deployment.yaml". Notice the password name, the persistentVolumeClaim, and the volumeMounts information. These entries should look familiar; we just created those things.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  selector:
    matchLabels:
      app: mysql
      tier: database
  template:
    metadata:
      labels:
        app: mysql
        tier: database
    spec:
      containers:
      - name: mariadb
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysqlpassword
```

```
            key: password
      image: mariadb
      resources:
        limits:
          memory: "128Mi"
          cpu: "500m"
      ports:
      - containerPort: 3306
      volumeMounts:
      - name: mysqlvolume
        mountPath: /var/lib/mysql
    volumes:
     - name: mysqlvolume
       persistentVolumeClaim:
         claimName: mysqlvolume
```

Copy snippet

We now have all the pieces to spin up a MariaDB database in our Kubernetes cluster. To get things going, run the following command:

```
kubectl create -f mysql-deployment.yaml
```

Copy snippet

Populating the database

Hint: You could put all of the following commands into a script.

The following instructions pertain to a PowerShell-based command line experience. For instructions using Bash Shell, scroll down further in this activity.

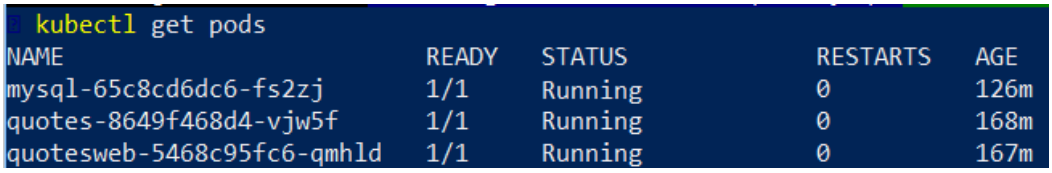
PowerShell:

We need the name of the pod running MariaDB. Note that the pod name begins with "mysql". Use the following command:

```
kubectl get pods
```

Copy snippet

For example:



This puts the pod name into a variable to be used in the remaining command. In this particulate example, the pod name is "mysql-65c8cd6dc6-fs2zj".

Use the name to run the following command, substituting the name of your MariaDB pod:

```
$podname="{mysql pod name from previous kubectl get pods command}"
```

Copy snippet

For example:

```
$podname="mysql-65c8cd6dc6-fs2zj"
```

Copy snippet

Copy the database creation commands into the pod and execute the script.

```
kubectl cp ./create_database_quotesdb.sql ${podname}:/tmp/create_database_quotesdb.sql
kubectl cp ./create_database.sh ${podname}:/tmp/create_database.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/create_database.sh
```

Copy snippet

Copy the table creating commands into the pod and execute the script.

```
kubectl cp ./create_table_quotes.sql ${podname}:/tmp/create_table_quotes.sql
kubectl cp ./create_tables.sh ${podname}:/tmp/create_tables.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/create_tables.sh
```

Copy snippet

Populate the table

```
kubectl cp ./populate_table_quotes_POWERSHELL.sql ${podname}:/tmp/populate_table_quotes_POWERSHELL.sql
kubectl cp ./quotes.csv ${podname}:/tmp/quotes.csv
kubectl cp ./populate_tables_POWERSHELL.sh ${podname}:/tmp/populate_tables_POWERSHELL.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/populate_tables_POWERSHELL.sh
```

Copy snippet

Query table to prove our work

```
kubectl cp ./query_table_quotes.sql ${podname}:/tmp/query_table_quotes.sql
kubectl cp ./query_table_quotes.sh ${podname}:/tmp/query_table_quotes.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/query_table_quotes.sh
```

Copy snippet

Bash:

We need the name of the pod running MariaDB. Note that the pod name begins with "mysql". Use the following command:

```
kubectl get pods
```

Copy snippet

For example:

```
kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
mysql-65c8cd6dc6-fs2zj        1/1     Running   0          126m
quotes-8649f468d4-vjw5f       1/1     Running   0          168m
quotesweb-5468c95fc6-qmhld    1/1     Running   0          167m
```

Use the name to run the following command, substituting the name of your MariaDB pod:

```
export PODNAME="{mysql pod name from previous kubectl get pods command}"
```

Copy snippet

For example:

```
export PODNAME="mysql-65c8cd6dc6-fs2zj"
```

Copy snippet

This puts the pod name into a variable to be used in the remaining command. In this particulate example, the pod name is "mysql-65c8cd6dc6-fs2zj".

Copy the database creation commands into the pod and execute the script.

```
kubectl cp ./create_database_quotesdb.sql $PODNAME:/tmp/create_database_quotesdb.sql
kubectl cp ./create_database.sh $PODNAME:/tmp/create_database.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/create_database.sh
```

Copy snippet

Copy the table creating commands into the pod and execute the script.

```
kubectl cp ./create_table_quotes.sql $PODNAME:/tmp/create_table_quotes.sql
kubectl cp ./create_tables.sh $PODNAME:/tmp/create_tables.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/create_tables.sh
```

Copy snippet

Populate the table

```
kubectl cp ./populate_table_quotes_BASH.sql $PODNAME:/tmp/populate_table_quotes_BASH.sql
kubectl cp ./quotes.csv $PODNAME:/tmp/quotes.csv
kubectl cp ./populate_tables_BASH.sh $PODNAME:/tmp/populate_tables_BASH.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/populate_tables_BASH.sh
```

Copy snippet

Query table to prove our work

```
kubectl cp ./query_table_quotes.sql $PODNAME:/tmp/query_table_quotes.sql
kubectl cp ./query_table_quotes.sh $PODNAME:/tmp/query_table_quotes.sh
kubectl exec deploy/mysql -- /bin/bash ./tmp/query_table_quotes.sh
```

Copy snippet

DevOps

Write these scripts. Automate. Then, save those scripts in the git repo with the project. Operations can then come along, duplicate what you've done, and improve on the scripts. You're a Developer. Working with Operations. DevOps.

Expose service

This will expose the MariaDB database as a service inside our Kubernetes cluster. If you don't do this, your other applications will not be able to discover this service. This entire concept of naming your services and making them visible to other services is known as "service discovery". Kubernetes can discover pods by a label that you assign instead of a random name or IP address.

Use this command to create the service we'll call "mysql":

```
kubectl expose deploy/mysql --name mysql --port 3306 --type NodePort
```

Copy snippet

We get to assign the name: In this case we're calling it "mysql". We can then reference it by this name from other applications. For example, the code to connect to the database in our "quotes" program, which is written in Python, is as follows:

```
        conn = mariadb.connect(
            user="root",
            password="admin",
            host=os.environ['DB_SERVICE_NAME'],
            database="quotesdb",
            port=3306)
```

Copy snippet

This is a terrible example of code. The user and password are here in clear text. Earlier, we used a Kubernetes Secret object when creating the database application. In real life, you would also create a secret for use by your Python code. Which is an outstanding assignment left to you, the reader. Hint: You need to import the Kubernetes client for Python to access Kubernetes secrets.

Update backend program (quotes) to Version 2

Before we switch to Version 2, we need to do some housekeeping. While Version 1 of our "quotes" service hard values hard-coded into the code, Version 2 reads from the database service "mysql". The name of this service is not hard-coded into the source code. Instead, it reads the name of the service from the environment variable "DB_SERVICE_NAME". Here's the code snippet where that happens:

```
    try:
        conn = mariadb.connect(
            user="root",
            password="admin",
            host=os.environ['DB_SERVICE_NAME'],
            database="quotesdb",
            port=3306)
```

Copy snippet

The following command will create that environment variable in our deployment. Then, when we update to Version 2, it will be available to the Python code:

```
kubectl set env deployment/quotes DB_SERVICE_NAME=mysql
```

At this point, we have a frontend application ("quotesweb") talking to the backend app ("quotes"). We also have a database, running in service "mysql". What we need to do is update our backend app to use our database.

Kubernetes will do this on the fly, doing what's called a "rolling update". We already have a Version 2 image in an image registry, so all we need to do is change the image in our deployment of "quotes" to point to Version 2. Kubernetes will pull the image, spin up a pod running Version 2, and then will switch the routing to Version 2.

To do this, simply use the following command; it's literally this easy:

```
kubectl set image deploy quotes quotes=quay.io/donschenck/quotes:v2
```

Copy snippet

After a few seconds – *seconds*, not minutes – you may need to refresh your browser. At that point, you will notice that there are several more quotes being randomly accessed.

Hint: What would happen if you switched back to v1?

What did you do?

You created a backend app. You created a frontend app, and connected the two.

You create a database app running in Kubernetes, and you populated it from your command line.

You scaled an application with one command.

You updated an application on the fly.

Destroying the MariaDB pod

Because we use a PVC for our database, instead of an ephemeral database, our data remains intact when a pod falls over. You can prove this by deleting the pod running your MariaDB database. Kubernetes will replace the pod immediately and MariaDB will restart. With no operator intervention. Go ahead, give it a try:

PowerShell:

```
kubectl delete pod ${podname}
```

Copy snippet

Bash:

```
kubectl delete pod $PODNAME
```

Copy snippet

You know Kubernetes

You know Kubernetes! Granted, it's just a start, but you're on your way. Congratulations. We have a virtual ton of information available at developers.redhat.com. I suggest you start at KubernetesByExample.com. See you there.

Related articles

A visual guide to deploying JBoss EAP on AWS

How Kamelets simplify Camel integrations on Kubernetes

How odo 3.0 GA improves the developer experience

Open vSwitch: The revalidator process explained

Tools for OpenShift Sandbox

My advice on why you should build containers on your PC

Hello World for Red Hat OpenShift Dev Spaces (formerly CodeReady Workspaces)

How to install command-line tools on a Mac

Developer tools rebrand, say farewell to CodeReady name

Introducing Red Hat OpenShift extension for Docker Desktop

Looking for more Kafka?

Learn more about the new Red Hat OpenShift Streams for Apache Kafka.

Learn more

©2022 Red Hat, Inc. | Preferências de cookies | Privacy Statement | Terms of Use | All policies and guidelines

Red Hat Summit